

Engine Allgemein

[Funktionsweise](#)

[Globales Feature: condition \(Bedingte Ausführung\)](#)

[Best Practices & Regeln für Variablennamen](#)

[Variablen-Auflösung](#)

[Wichtige YAML-Syntaxregeln](#)

Funktionsweise

Die Engine folgt einem "**Decoupled Execution Model**": Ein Befehl wird ausgeführt und der gesamte Output wird im Arbeitsspeicher unter einem frei wählbaren Namen (`response`) zwischengespeichert. Dies erlaubt beliebig viele Prüfungen (ASSERT) oder Datenextraktionen (BUFFER) auf demselben Output, ohne den Befehl erneut auszuführen.

Globales Feature: `condition` (Bedingte Ausführung)

Mit dem optionalen Feld `condition` steuern Sie flexibel, ob ein Testschritt ausgeführt werden soll. Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt übersprungen (`SKIPPED`) und der Testlauf ohne Fehler fortgesetzt.

Das Framework führt vor der Evaluierung eine automatische Zusammenführung aller Variablen und Buffer-Inhalte durch, sodass die Bedingung auf den aktuellen Zustand des Systems prüfen kann.

Syntax-Hinweis: Da Werte direkt evaluiert werden, müssen Strings in einfache Anführungszeichen gesetzt werden (`'...'`).

YAML-Beispiel:

```
1 | YAML
2 |
3 | # Der Schritt wird nur ausgeführt, wenn der in einem vorherigen Schritt
4 | # im Buffer gespeicherte Wert 'os_version_var' dem String 'Windows_NT' e
5 | - type: CMD
6 |   condition: "'{B[os_version_var]}' == 'Windows_NT'"
7 |   op: EXEC
8 |   command: "{B[command]}"
9 |   response: windows_version
```

! WICHTIGER WARNHINWEIS FÜR AGATE-ANWENDER Obwohl die AGATE-Testsuite intern fortschrittliche Mechanismen nutzt, um kritische Sonderzeichen in Bedingungen (wie Bindestriche `-`, Klammern `()` oder führende Zahlen) abzufangen und zu bereinigen, sollten AGATE-Anwender diese "Provokationen" bei der Namensgebung von Variablen strikt vermeiden. Unsaubere Variablennamen (die z. B. oft durch automatisierte Tosca-Exporte entstehen) können die mathematische Evaluierungs-Engine (JEXL) verwirren und zu unerwarteten `true / false`-Ergebnissen führen.

Best Practices & Regeln für Variablennamen

Um eine reibungslose und fehlerfreie Ausführung der `condition`-Logik zu gewährleisten, müssen Variablennamen (inkl. Keys aus CSV-Dateien und Buffern) folgenden Regeln entsprechen:

1. **Erlaubte Zeichen:** Verwenden Sie ausschließlich alphanumerische Zeichen (A-Z, a-z, 0-9) und den Unterstrich (`_`).
2. **Erstes Zeichen:** Eine Variable **muss immer** mit einem Buchstaben oder einem Unterstrich beginnen. Sie darf **niemals mit einer Zahl** beginnen (z. B. `3_test` ist unzulässig).
3. **Keine mathematischen Operatoren:** Vermeiden Sie Bindestriche/Minuszeichen (`-`), Pluszeichen (`+`) oder Schrägstriche (`/`). Ein Name wie `CardToken.SVNR-Karte` wird von der Engine fälschlicherweise als Subtraktion interpretiert. Nutzen Sie stattdessen den Unterstrich (z. B. `CardToken_SVNR_Karte`).
4. **Keine Sonder- und Gruppierungszeichen:** Zeichen wie Klammern (`()`), eckige Klammern (`[]`), Fragezeichen (`?`) oder At-Zeichen (`@`) sind absolut tabu (z. B. Variablennamen wie `(1)` oder `Kunde@Daten` zerstören die Syntax der Bedingung).

Variablen-Auflösung

Alle Dynamic-Tags (`{B[...]}`, `{R[...]}`, `{T[...]}`, `{E[...]}`) werden im `allen` Felder unterstützt bzw. resolved.

Wichtige YAML-Syntaxregeln

Das Schreiben von Systembefehlen in YAML erfordert Aufmerksamkeit bei Sonderzeichen:

1. **Geschweifte Klammern** `{ }` : Da sie ein Inline-Objekt einleiten, müssen Platzhalter wie `{B[var]}` immer in Anführungszeichen gesetzt werden (`command: "{B[var]}"`).

2. **Doppelpunkt und Backslashes:** Da er ein Trennzeichen ist, müssen Pfade oder URLs mit Doppelpunkten in Anführungszeichen stehen (`command: "C:\\Tools\\run.bat"`). Da Backslashes in YAML als Escape-Zeichen dienen, achten Sie bei der Angabe von absoluten Pfaden auf doppelte Backslashes: `command: "C:\\Scripts\\my_tool.bat"`

3. **Anführungszeichen:**

- **Option A:** Umschließe den String mit einfachen Anführungszeichen, um doppelte innen nutzen zu können: `command: 'echo "Hallo"'` .
- **Option B:** Escape doppelte Anführungszeichen mit Backslash: `command: "echo \\\"Hallo\\\""` .

4. **Mehrzeilige Befehle:** Nutze die Block-Notation `|` , um komplexe Skripte sauber abzubilden:

```
1 | YAML
```

```
1 - type: CMD
2   op: EXEC
3   command: |
4     cd C:\Services\App
5     run_install.bat --mode=silent --dir="C:\Program Files\App"
6   response: install_out
```